

HarmonyOS ArkTS + Flask + SQLite

教学过程管理系统

项目使用说明

面向前端开发、后端开发、测试与演示人员的完整操作手册
包含项目结构、环境配置、Flask 启动、数据库准备、前后端联调、
角色功能、测试与故障排查

代码仓库:	https://github.com/2024080911015/harmonyOS_teaching_manage
分支:	main
文档依据提交:	645bbabceef218377899fe965842c04396ff817a
提交说明:	基本流程跑通
文档日期:	2026-07-14

文档说明

本说明基于仓库 main 分支提交 645bbab 的实际代码编写，覆盖当前根目录下的 HarmonyOS 主工程 entry/ 与 Flask 服务端 teaching_flask_server/。文档不是抽象设计稿，而是针对当前初版项目的可执行使用指南。

先明确一个仓库结构问题

仓库根目录除主工程外，还存在一个 1/ 子目录，内部包含另一套可以独立打开的 HarmonyOS 工程。根目录的 build-profile.json5 明确将活动模块指向 ./entry，因此：

- 在 DevEco Studio 中打开仓库根目录时，实际编译的是根目录 entry/；
- 1/entry/ 不会自动参与根目录工程编译；
- 日常开发不要在两套目录之间混合修改，否则容易出现“代码改了但运行结果没变”的问题；
- 本文默认使用根目录主工程，1/ 仅作为历史版本或独立原型参考。

当前项目状态

- 前端：HarmonyOS ArkTS，根入口页面为 pages/Index.ets；
- 后端：Flask 应用工厂 + Blueprint；
- 数据库：SQLite，默认文件为 teaching_flask_server/data/teaching.db；
- 认证：账号密码登录 + Bearer Token；
- 角色：学生、教师、教务人员、管理员；
- 自动化测试：仓库报告记录 78 条 Flask 路由、99 个测试用例全部通过；
- 当前定位：适合课程项目、本地联调和演示，不应直接作为生产环境部署方案。

目录

文档说明	i
第一章 项目概览	1
1.1 项目定位	1
1.2 角色与主要能力	1
1.3 总体技术架构	1
1.4 统一数据约定	2
1.4.1 统一响应格式	2
1.4.2 鉴权请求头	3
第二章 五分钟快速启动	4
2.1 准备环境	4
2.2 获取代码	4
2.3 启动 Flask 服务端	4
2.4 配置 HarmonyOS 端地址	5
2.5 运行 HarmonyOS 应用	6
2.6 体验账号	6
第三章 仓库与目录结构	8
3.1 根目录结构	8
3.2 HarmonyOS 主模块结构	8
3.2.1 页面注册方式	9
3.3 API 封装结构	9
3.4 Flask 服务端结构	10
第四章 HarmonyOS 前端运行与配置	12
4.1 打开正确的工程	12
4.2 构建配置	12
4.3 网络权限	12

4.4	应用启动流程	13
4.5	登录与会话恢复	13
4.6	前端调用 API 的规则	13
4.7	模拟器和真机联调	14
第五章	Flask 服务端完整启动说明	15
5.1	依赖与虚拟环境	15
5.1.1	PowerShell 推荐流程	15
5.1.2	CMD 流程	15
5.2	数据库位置与环境变量	15
5.3	首次或更新后执行迁移	16
5.4	创建首个管理员	16
5.5	生成测试数据	17
5.6	启动服务	18
5.7	验证服务	18
5.7.1	浏览器或 PowerShell 健康检查	18
5.7.2	PowerShell 登录测试	18
5.8	停止服务	19
5.9	生产环境说明	19
第六章	数据库与业务数据	20
6.1	数据库连接策略	20
6.2	主要数据表分组	20
6.3	事务规则	21
6.4	权威数据与本地数据	21
第七章	前端页面与用户操作	22
7.1	登录页与主容器: Index.ets	22
7.2	首页: HomePage.ets	22
7.3	课表: TimetablePage.ets	22
7.4	课程: CoursePage.ets	23
7.4.1	学生视角	23
7.4.2	教师视角	23
7.5	事项办理与审批: WorkflowPage.ets	23
7.5.1	学生	23
7.5.2	教师	23
7.5.3	教务和管理员	24

7.6 消息: NotificationPage.ets	24
7.7 教学管理: AdminPage.ets	24
7.8 个人中心: ProfilePage.ets	24
第八章 API 封装与前端调用	26
8.1 统一 Api 实例	26
8.2 业务模块速查	26
8.3 常用调用示例	27
8.3.1 登录	27
8.3.2 课程分页	27
8.3.3 提交教室申请	27
8.3.4 提交请假	27
8.3.5 审批	28
8.4 QueryParams 机制	28
8.5 401 处理	28
第九章 Flask 模块与接口概览	29
9.1 认证 auth.py	29
9.2 个人资料 profile.py	29
9.3 基础数据 common.py	29
9.4 课程 courses.py	30
9.5 通知 notifications.py	30
9.6 教室预约 bookings.py	31
9.7 请假 leaves.py	31
9.8 调课 schedule_changes.py	31
9.9 管理 admin.py	32
第十章 关键业务流程与规则	33
10.1 登录流程	33
10.2 选课流程	33
10.3 教室预约流程	34
10.4 请假流程	34
10.5 调课流程	34
10.6 通知流程	34
第十一章 测试与验收	35
11.1 运行全接口测试	35

11.2 前端冒烟测试	36
11.3 DevEco 构建验收	36
第十二章 常见问题与排查	37
12.1 无法连接 Flask	37
12.2 登录提示网络失败	37
12.3 提示 SessionStore 尚未初始化	38
12.4 修改代码但应用没变化	38
12.5 端口 8080 被占用	38
12.6 数据库被锁定	38
12.7 JSON 字段为空或类型不对	38
第十三章 当前初版的已知限制与改进建议	39
13.1 仓库存在两套 HarmonyOS 工程	39
13.2 ApiConfig 写死局域网 IP	39
13.3 ApiClient 的请求头类型	39
13.4 ApiError 未保留冲突详情	39
13.5 教师请假查看页面尚未接入	40
13.6 管理页面仍偏演示型	40
13.7 初始化失败没有备用页面	40
13.8 后端开发模式不适合生产	40
第十四章 团队开发建议	41
14.1 推荐分工边界	41
14.2 修改接口的标准流程	41
14.3 提交前检查清单	42
附录 A 常用命令速查	43
A.1 后端	43
A.2 Windows 网络	43
附录 B 角色功能验收表	45
附录 C 文档维护说明	46

项目概览

1.1 项目定位

本项目用于统一管理教学过程中的常见业务，核心目标是让学生、教师、教务人员和管理员在一个移动端工作台完成课程、课表、通知、教室、请假、调课与审批操作。

系统采用前后端分离架构：HarmonyOS 应用只负责界面、交互和调用 API；Flask 服务端负责认证、权限、业务规则、事务与数据库访问；SQLite 保存所有权威业务数据。

1.2 角色与主要能力

角色	主要页面	当前能力
STUDENT 学生	首页、课表、课程、事项办理、消息、个人中心	查看和选择课程、退课、查看个人课表、申请教室、提交请假、查看通知、修改资料和密码
TEACHER 教师	首页、课表、授课课程、事项办理、消息、个人中心	查看授课课程及学生名单、查看课表、申请教室、提交调课、发布和撤回本人通知、修改资料和密码
ACADEMIC_STAFF 教务人员	首页、课表、课程、审批中心、消息、教学管理、个人中心	审批教室、请假、调课；管理人员和基础数据；管理通知；查看审批记录
ADMIN 管理员	与教务人员相同，并具有更高账号管理权限	创建高权限账号、停用账号、重置密码、管理全部基础数据和通知

1.3 总体技术架构



```
      v
Api.ets 统一入口
      |
      v
ApiClient.ets + SessionStore.ets
      |
      | HTTP / JSON / Bearer Token
      v
Flask app.py
      |
      v
auth_guard.py 认证与角色校验
      |
      v
blueprints/*.py 业务路由
      |
      v
services.py 冲突检查和课表计算
      |
      v
db.py -> SQLite data/teaching.db
```

1.4 统一数据约定

1.4.1 统一响应格式

Flask 所有业务接口统一返回：

```
{
  "code": 0,
  "message": "success",
  "data": {}
}
```

其中：

- code = 0 表示业务成功；
- 非零 code 表示参数、认证、权限、冲突或服务器错误；
- ArkTS 的 ApiClient.ets 会解析该结构，成功时只把 data 返回给页面；
- 页面不需要重复判断 HTTP 响应和业务码。

1.4.2 鉴权请求头

除健康检查和登录外，大部分接口需要：

```
Authorization: Bearer <token>  
Content-Type: application/json  
Accept: application/json
```

Token 由登录接口生成并写入数据库 auth_token，HarmonyOS 端通过 Preferences 保存当前会话。

五分钟快速启动

本章给出最短可执行路径。第一次接触项目时，先按本章完成端到端运行，再阅读后续结构说明。

2.1 准备环境

建议准备：

- Windows 10/11;
- DevEco Studio，并安装与项目兼容的 HarmonyOS SDK;
- Python 3.10 或更高版本；仓库测试报告使用过 Python 3.13.6;
- Git;
- DevEco Studio 模拟器或 HarmonyOS 真机。

根工程配置的目标和兼容 SDK 为 6.1.1(24)。若 DevEco Studio 提示缺少 SDK，应在 SDK Manager 安装相应版本，或由项目负责人统一调整版本配置。

2.2 获取代码

```
git clone https://github.com/2024080911015/harmonyOS_teaching_manage.git
cd harmonyOS_teaching_manage
```

在 DevEco Studio 中选择仓库根目录，不要默认打开 1/ 子目录。

2.3 启动 Flask 服务端

在新终端中执行：

```
cd teaching_flask_server
python -m venv .venv
```

```
.\.venv\Scripts\Activate.ps1  
python -m pip install -r requirements.txt  
python migrate.py  
python app.py
```

若 PowerShell 禁止执行激活脚本，可以临时执行：

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
.\.venv\Scripts\Activate.ps1
```

如果使用传统 CMD：

```
.\venv\Scripts\activate.bat
```

服务默认监听：

```
http://0.0.0.0:8080
```

电脑本机测试地址：

```
http://127.0.0.1:8080/api/health
```

预期响应：

```
{  
  "code": 0,  
  "message": "success",  
  "data": { "status": "UP" }  
}
```

2.4 配置 HarmonyOS 端地址

打开：

```
entry/src/main/ets/api/core/ApiConfig.ets
```

当前仓库中写的是类似：

```
export class ApiConfig {
  static readonly BASE_URL: string =
    'http://192.168.54.22:8080/api'

  static readonly CONNECT_TIMEOUT: number = 10000
  static readonly READ_TIMEOUT: number = 15000
}
```

把 IP 改成运行 Flask 的电脑局域网 IPv4 地址。Windows 中查询：

```
ipconfig
```

找到当前联网网卡的 IPv4 地址，例如 192.168.1.105，再修改为：

```
'http://192.168.1.105:8080/api'
```

不要在模拟器中写 127.0.0.1

模拟器里的 127.0.0.1 指向模拟器自己，不是运行 Flask 的 Windows 电脑。DevEco Studio 模拟器和真机联调时，优先使用电脑局域网 IP，并确保 Windows 防火墙允许 Python 或 TCP 8080 端口。

2.5 运行 HarmonyOS 应用

在 DevEco Studio 中：

- 1. 等待项目同步和索引完成；
- 2. 选择根模块 entry；
- 3. 启动模拟器；
- 4. 点击 Run；
- 5. 应用进入登录页后，选择体验账号或手动输入账号。

2.6 体验账号

仓库自带测试数据时，可以使用：

角色	账号示例	密码	用途
----	------	----	----

学生	test_student_0001	Test123456	选课、请假、教室申请
教师	test_teacher_001	Test123456	授课、调课、发布通知
教务	test_staff_001	Test123456	审批和基础管理
管理员	test_admin_01	Test123456	完整管理功能

快速启动完成标准

- 浏览器访问 /api/health 成功；
- DevEco 模拟器能打开登录页；
- 使用测试账号登录成功；
- 首页可以显示统计与通知；
- 课程、课表、办理页面可以读取数据。

仓库与目录结构

3.1 根目录结构

```

harmonyOS_teaching_manage/
├─ AppScope/                # HarmonyOS 应用级配置与资源
├─ entry/                    # 当前根工程实际编译的主模块
├─ teaching_flask_server/    # Flask + SQLite 服务端
├─ 1/                        # 另一套独立 HarmonyOS 工程/历史原型
├─ build-profile.json5       # 根工程构建配置，指向 ./entry
├─ hvigorfile.ts             # Hvigor 构建入口
├─ oh-package.json5          # 根工程依赖配置
├─ code-linter.json5         # ArkTS 代码检查配置
└─ ...

```

3.2 HarmonyOS 主模块结构

```

entry/src/main/
├─ ets/
│   ├─ entryability/
│   │   └─ EntryAbility.ets    # 应用生命周期与 API 初始化
│   ├─ entrybackupability/
│   │   └─ EntryBackupAbility.ets
│   └─ api/
│       ├─ Api.ets             # 全项目统一 API 实例
│       ├─ core/               # 网络、会话、错误与公共类型
│       ├─ model/              # 请求/响应数据模型
│       └─ module/             # 各业务 API 模块
├─ components/
│   └─ UiKit.ets               # 通用 UI 组件
├─ pages/
│   ├─ Index.ets               # 登录、会话恢复、底部导航
│   └─ HomePage.ets            # 角色首页

```

```

|   |   |— TImetablePage.ets    # 个人课表
|   |   |— CoursePage.ets      # 课程、选课、教学名单
|   |   |— WorkflowPage.ets    # 申请和审批
|   |   |— NotificationPage.ets # 消息与通知发布
|   |   |— AdminPage.ets       # 人员和教学管理
|   |   |— ProfilePage.ets     # 个人中心
|   |   └─ utils/
|   |       └─ UiUtils.ets      # Toast、状态文案、日期工具
|— module.json5                # 模块、Ability、网络权限
└─ resources/
    └─ base/profile/main_pages.json

```

3.2.1 页面注册方式

main_pages.json 当前只注册：

```

{
  "src": ["pages/Index"]
}

```

其余页面并不是独立路由页面，而是由 Index.ets 作为组件导入，并通过 currentTab 切换显示。因此：

- 当前项目不依赖多页面路由跳转；
- 底部导航在 Index.ets 中统一控制；
- 需要新增主标签页时，应同时修改组件导入、标签索引和底部导航逻辑。

3.3 API 封装结构

```

api/
├─ Api.ets
├─ core/
|   ├─ ApiConfig.ets
|   ├─ ApiTypes.ets
|   ├─ ApiError.ets
|   ├─ QueryBuilder.ets
|   ├─ SessionStore.ets
|   └─ ApiClient.ets
├─ model/
|   ├─ CommonModel.ets
|   ├─ AuthModel.ets
|   └─ ProfileModel.ets

```

```
|   ├── NotificationModel.ets
|   ├── CourseModel.ets
|   ├── BookingModel.ets
|   ├── LeaveModel.ets
|   ├── ScheduleChangeModel.ets
|   └── AdminModel.ets
└── module/
    ├── AuthApi.ets
    ├── ProfileApi.ets
    ├── CommonApi.ets
    ├── NotificationApi.ets
    ├── CourseApi.ets
    ├── BookingApi.ets
    ├── LeaveApi.ets
    ├── ScheduleChangeApi.ets
    └── AdminApi.ets
```

各层职责如下：

文件或目录	职责
Api.ets	创建唯一 ApiClient，实例化所有业务模块，初始化 SessionStore，向页面暴露统一 Api 对象
ApiConfig.ets	保存 Flask 基础地址和超时时间
ApiTypes.ets	统一响应、分页、查询参数、角色与状态枚举
ApiError.ets	统一网络错误、HTTP 错误和 Flask 业务错误
QueryBuilder.ets	将查询参数构造为 URL 查询字符串
SessionStore.ets	使用 Preferences 保存 Token、用户 ID、账号、姓名、角色与过期时间
ApiClient.ets	创建 HTTP 请求、自动携带 Token、解析 JSON、处理 401、销毁请求对象
model/	只声明请求和响应结构，不发送网络请求
module/	按业务封装路径和调用方法，供页面直接使用

3.4 Flask 服务端结构

```
teaching_flask_server/
├── app.py           # 应用工厂、Blueprint 注册、健康检查
└── config.py       # 数据库、Token、请求体大小配置
```

```
|— db.py                # SQLite 连接与请求结束自动关闭
|— api_utils.py         # 统一响应、camelCase、参数解析、分页
|— auth_guard.py        # Bearer Token 与角色校验
|— services.py         # 课表计算和冲突检查
|— migrate.py          # 可重复数据库迁移
|— create_admin.py     # 创建首个管理员
|— seed_data.py        # 生成完整测试数据
|— test_all_api.py     # 全接口自动化测试
|— requirements.txt    # Python 依赖
|— blueprints/
|   |— auth.py
|   |— profile.py
|   |— common.py
|   |— notifications.py
|   |— courses.py
|   |— bookings.py
|   |— leaves.py
|   |— schedule_changes.py
|   |— admin.py
|— data/
|   |— teaching.db
|   |— test_data_summary.json
```

HarmonyOS 前端运行与配置

4.1 打开正确的工程

在 DevEco Studio 使用 **Open** 打开仓库根目录。打开后应看到根目录 entry/ 被识别为主模块。若工程窗口只显示 1/entry/，说明打开了嵌套工程，应关闭后重新打开仓库根目录。

4.2 构建配置

根目录 build-profile.json5 中：

- 产品名为 default；
- targetSdkVersion 为 6.1.1(24)；
- compatibleSdkVersion 为 6.1.1(24)；
- 运行系统为 HarmonyOS；
- 模块路径为 ./entry；
- 开启大小写敏感与规范化 OHM URL 检查。

4.3 网络权限

entry/src/main/module.json5 已声明：

```
"requestPermissions": [  
  {  
    "name": "ohos.permission.INTERNET"  
  }  
]
```

若合并代码时误删该权限，API 会初始化成功，但实际 HTTP 请求会失败。

4.4 应用启动流程

EntryAbility.ets 的关键流程为：

```
onWindowStageCreate()  
-> initializeApiAndLoadPage()  
-> await Api.init(this.context)  
-> SessionStore.init(context)  
-> windowStage.loadContent('pages/Index')
```

页面加载前必须初始化 Preferences，否则 ApiClient 读取 Token 时会抛出“Session-Store 尚未初始化”。

4.5 登录与会话恢复

Index.ets 在 aboutToAppear() 中：

1. 注册 Token 失效回调；
2. 调用 Api.hasSession() 检查本地是否存在 Token；
3. 存在 Token 时调用 Api.auth.me() 让服务端验证；
4. 验证成功后恢复姓名、角色和账号；
5. 验证失败则清除本地会话并显示登录页。

这比只检查本地 Token 更可靠，因为 Token 可能已经过期、被撤销或账号已被停用。

4.6 前端调用 API 的规则

页面只导入统一入口：

```
import { Api } from '../api/Api'
```

示例：

```
const page = await Api.course.getCourses({  
  page: 1,  
  pageSize: 20,  
  keyword: '数据库'  
})  
  
const courses = page.items
```

错误处理:

```
try {
  await Api.course.enroll(courseId)
} catch (error) {
  showToast(this.getUIContext(), {
    message: errorMessage(error as Object)
  })
}
```

页面不应:

- 直接写 Flask 完整 URL;
- 自己创建 http.createHttp();
- 手动读取或拼接 Token;
- 自己解析 code/message/data;
- 在多个页面中各自创建新的 ApiClient。

4.7 模拟器和真机联调

场景	ApiConfig 地址	注意事项
电脑浏览器访问后端	http://127.0.0.1:8080/api	仅限电脑本机工具和浏览器
DevEco 模拟器访问电脑后端	http://□□□□IP:8080/api	不要使用模拟器的 127.0.0.1
HarmonyOS 真机访问电脑后端	http://□□□□IP:8080/api	手机和电脑应在同一局域网
远程服务器部署	https://□□□□/api	应启用 HTTPS、反向代理和生产 WSGI

Flask 服务端完整启动说明

5.1 依赖与虚拟环境

服务端当前 requirements.txt 仅声明：

```
Flask>=3.0,<4.0
```

Flask 会安装 Werkzeug 等依赖。建议使用项目独立虚拟环境，避免全局 Python 包冲突。

5.1.1 PowerShell 推荐流程

```
cd teaching_flask_server
python -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

5.1.2 CMD 流程

```
cd teaching_flask_server
python -m venv .venv
.venv\Scripts\activate.bat
python -m pip install -r requirements.txt
```

5.2 数据库位置与环境变量

默认数据库路径由 config.py 决定：

```
teaching_flask_server/data/teaching.db
```

可通过 PowerShell 环境变量替换：

```
$env:TEACHING_DB = "D:\data\teaching.db"
$env:TOKEN_EXPIRE_DAYS = "7"
python app.py
```

主要配置：

配置	默认值	含义
TEACHING_DB	data/teaching.db	SQLite 数据库文件路径
TOKEN_EXPIRE_DAYS	7	登录 Token 有效天数
MAX_CONTENT_LENGTH	8 MB	Flask 请求体最大大小

5.3 首次或更新后执行迁移

```
python migrate.py
```

迁移脚本可以重复执行，主要补充或维护：

- semester 学期表；
- notification.expires_at;
- leave_request_course;
- room_booking_log;
- 查询索引；
- 考虑单双周的选课冲突触发器。

执行迁移前先备份

虽然当前 migrate.py 设计为可重复执行，但数据库属于核心业务数据。正式演示或多人共享数据库执行结构修改前，应复制一份 teaching.db 作为备份。

5.4 创建首个管理员

当使用空数据库或数据库中没有管理员时：

```
python create_admin.py \  
  --account admin \  
  --password Admin123456 \  
  --name 系统管理员
```

可选指定工号：

```
python create_admin.py \  
  --account admin \  
  --password Admin123456 \  
  --name 系统管理员 \  
  --employee-no ADMIN001
```

该脚本会同时创建 `sys_user` 和 `academic_staff_profile` 记录。项目不提供公开注册，其他账号由管理员接口预先创建。

5.5 生成测试数据

仓库当前已包含一份测试数据库和摘要。需要重新生成时，可执行：

```
python seed_data.py --users 2000 --password Test123456 --replace
```

常用参数：

- `--users`：账号总数，默认 2000，不能小于 100；
- `--password`：测试账号统一密码，默认 Test123456；
- `--seed`：随机种子；
- `--replace`：清空现有业务数据并重新生成。

-replace 会清空业务数据

`seed_data.py --replace` 会删除现有用户、课程、选课、预约、请假、调课、通知等业务数据。不要对需要保留的数据库执行。没有 `--replace` 时，脚本发现已有用户会主动终止，以保护现有数据。

当前测试摘要记录的数据规模包括：2000 个用户、240 门课程、240 条基础排课、10937 条选课记录、500 条教室预约、600 条请假、180 条调课、120 条通知等。

5.6 启动服务

```
python app.py
```

app.py 直接运行时等价于：

```
create_app().run(  
    host="0.0.0.0",  
    port=8080,  
    debug=True  
)
```

0.0.0.0 表示监听全部网卡，并不是客户端应输入的地址。客户端实际使用：

- 电脑本机：127.0.0.1:8080；
- 模拟器和真机：电脑局域网 IP 加 :8080。

5.7 验证服务

5.7.1 浏览器或 PowerShell 健康检查

```
Invoke-RestMethod \  
-Method Get \  
-Uri http://127.0.0.1:8080/api/health
```

5.7.2 PowerShell 登录测试

```
$body = @{  
    account = "test_student_0001"  
    password = "Test123456"  
} | ConvertTo-Json  
  
$result = Invoke-RestMethod \  
-Method Post \  
-Uri http://127.0.0.1:8080/api/auth/login \  
-ContentType "application/json" \  
-Body $body  
  
$result
```

5.8 停止服务

运行 Flask 的终端中按：

```
Ctrl + C
```

退出虚拟环境：

```
deactivate
```

5.9 生产环境说明

当前 debug=True 只适合本地开发。正式部署至少应：

- 关闭 debug；
- 使用 Waitress、Gunicorn 等生产 WSGI 服务；
- 使用 Nginx 或其他反向代理；
- 使用 HTTPS；
- 将数据库和密钥放在受控目录；
- 配置日志、备份和访问控制。

数据库与业务数据

6.1 数据库连接策略

db.py 的主要策略：

- 每个 Flask 请求第一次访问数据库时创建连接；
- 连接保存在 Flask g 对象；
- 使用 sqlite3.Row 支持按列名读取；
- 开启外键约束；
- 设置 15 秒 busy timeout；
- 请求结束自动关闭连接。

6.2 主要数据表分组

业务域	主要表
用户与认证	sys_user、auth_token
角色资料	student_profile、teacher_profile、 academic_staff_profile
基础数据	department、class_group、classroom、semester
课程教学	course、course_schedule、course_enrollment
教室预约	room_booking、room_booking_log
请假	leave_request、leave_request_course
调课	schedule_change
通知	notification、notification_recipient
审批审计	approval_record

6.3 事务规则

涉及多表写入的操作通常使用：

```
BEGIN IMMEDIATE
-> 更新业务主表
-> 写入审批记录或操作日志
-> commit
失败 -> rollback
```

例如教室审批需要同时更新预约状态、写审批记录和写预约日志；请假审批和调课审批也需要保持主记录与审批审计一致。

6.4 权威数据与本地数据

位置	保存内容	是否权威
SQLite 服务端	用户、课程、选课、课表、通知、预约、请假、调课、审批	是，业务事实以数据库为准
HarmonyOS Preferences	Token、用户 ID、账号、姓名、角色、过期时间	否，仅用于本地会话恢复
HarmonyOS 页面状态	加载状态、筛选条件、表单输入、当前标签页	否，应用退出后通常不保留

前端本地角色只用于页面展示，最终权限必须由 Flask 的 auth_guard.py 校验。

前端页面与用户操作

7.1 登录页与主容器：Index.ets

Index.ets 同时承担：

- 应用启动后的会话恢复；
- 登录表单和体验账号填充；
- Token 失效后的退出登录状态；
- 主页面组件切换；
- 底部导航栏。

登录成功后，普通用户看到：首页、课表、课程、办理、消息、我的。教务和管理员额外看到“管理”。

7.2 首页：HomePage.ets

首页调用：

- `Api.profile.getHome()` 获取角色资料 and 统计；
- `Api.notification.getReceived()` 获取最近三条通知。

角色统计：

- 学生：已选课程、待审教室、待申请假、未读通知；
- 教师：授课课程、待审教室、待审调课、未读通知；
- 教务/管理员：待审教室、待申请假、待审调课、未读通知。

7.3 课表：TimetablePage.ets

用户输入 YYYY-MM-DD 日期，前端调用：

```
Api.common.getMyTimetable(date)
```

后端根据学期起止日期计算教学周，再叠加审批通过的调课结果。页面展示实际生效的星期、节次和教室，并用“已调课”状态标识变更项。

7.4 课程：CoursePage.ets

7.4.1 学生视角

- “全部课程”：分页查询课程，支持关键字搜索；
- “我的课程”：查看选课记录；
- 可在开放时间且有余量时选课；
- 有效选课记录可执行退课。

7.4.2 教师视角

- 查询本人授课课程；
- 查看课程已选人数；
- 展开课程学生名单。

7.5 事项办理与审批：WorkflowPage.ets

7.5.1 学生

- 教室申请：选择教室、日期、节次、人数和用途；
- 请假申请：选择类型、日期、原因和受影响课程；
- 查看申请状态并取消允许取消的申请。

7.5.2 教师

- 提交教室申请；
- 针对本人基础排课提交 TIME、ROOM 或 BOTH 调课；
- 查看和取消本人待审批调课。

7.5.3 教务和管理员

- 审批教室申请；
- 审批学生请假；
- 审批教师调课；
- 驳回时填写审批意见。

7.6 消息：NotificationPage.ets

- 所有角色可查看收到的通知并标记已读；
- 教师可查看本人发布的通知；
- 教务和管理员可查看通知管理列表；
- 教师、教务和管理员可发布通知；
- 发布范围支持全体学生、指定班级、指定院系、指定用户；
- 教师只能在截止时间前撤回本人通知；
- 教务和管理员可以管理更广范围的通知。

当前发布表单要求直接填写目标 ID。后续产品化时可将其改为院系、班级和用户选择器。

7.7 教学管理：AdminPage.ets

该页面仅对教务人员和管理员显示，当前包含四个标签：

1. 人员：查询、创建、启停账号、重置密码；
2. 基础数据：创建院系、班级、教室和学期；
3. 课程排课：创建课程、创建排课、切换课程状态；
4. 审批记录：查看统一审批历史。

管理员和教务权限并不完全相同。后端会限制教务人员创建或修改其他教务/管理员账号。

7.8 个人中心：ProfilePage.ets

- 查看账号、角色、院系、班级、联系方式和最近登录时间；
- 修改性别、电话和邮箱；
- 教师还可修改职称与简介；

- 修改登录密码；
- 退出登录并清除本地会话。

API 封装与前端调用

8.1 统一 Api 实例

Api.ets 创建一个 TeachingApi 实例:

```
export const Api: TeachingApi = new TeachingApi()
```

所有模块共享同一个 ApiClient, 保证 Token、超时和 401 处理一致。

8.2 业务模块速查

模块	主要方法
Api.auth	login、me、logout、changePassword
Api.profile	getMyProfile、updateMyProfile、getHome
Api.common	health、getDepartments、getClasses、getSemesters、getClassrooms、getAvailableClassrooms、getMyTimetable
Api.notification	getReceived、getUnreadCount、markRead、publish、withdraw
Api.course	getCourses、getCourse、getMyEnrollments、enroll、drop、getTeachingCourses、getCourseStudents、getTeachingSchedules
Api.booking	create、getMyBookings、cancel、getLogs、getPending、approve、reject
Api.leave	create、getMyLeaves、getTeachingLeaves、getPending、approve、reject、cancel

Api.scheduleChange	create、getMyChanges、getConflicts、getPending、approve、reject、cancel
Api.admin	用户、院系、班级、教室、学期、课程、排课和审批记录管理

8.3 常用调用示例

8.3.1 登录

```
const result = await Api.auth.login(  
  'test_student_0001',  
  'Test123456'  
)
```

登录模块会自动将 Token 和用户信息保存到 SessionStore。

8.3.2 课程分页

```
const page = await Api.course.getCourses({  
  page: 1,  
  pageSize: 20,  
  keyword: '计算机'  
})  
  
console.info(`${page.total}`)  
console.info(JSON.stringify(page.items))
```

8.3.3 提交教室申请

```
await Api.booking.create({  
  classroomId: 10,  
  bookingDate: '2026-09-10',  
  startSection: 3,  
  endSection: 4,  
  purpose: '课程项目研讨',  
  participantCount: 30  
})
```

8.3.4 提交请假

```
await Api.leave.create({
```

```
leaveType: LeaveType.SICK,  
startDate: '2026-09-10',  
endDate: '2026-09-10',  
reason: '身体不适',  
courseIds: [2, 5]  
})
```

8.3.5 审批

```
await Api.booking.approve(bookingId, '教室可用, 同意申请')  
await Api.leave.reject(leaveId, '请补充相关材料')  
await Api.scheduleChange.approve(changeId, '冲突检查通过')
```

8.4 QueryParams 机制

ArkTS 对动态对象字面量检查严格，项目使用显式 QueryParams 类：

```
const params: QueryParams = new QueryParams()  
params.set('page', query.page)  
params.set('pageSize', query.pageSize)  
params.set('status', query.status)
```

null 和 undefined 会被忽略，但 0 和 false 会保留。

8.5 401 处理

当需要鉴权的接口返回 HTTP 401 或 401xx 业务码时，ApiClient 会：

1. 清除 Preferences 中的会话；
2. 调用 UnauthorizedHandler；
3. 抛出 ApiError；
4. Index.ets 将界面切回登录状态。

Flask 模块与接口概览

本章是接口导航，不替代具体 Model 类型。前端优先调用 ArkTS Api 模块，不直接拼接路径。

9.1 认证 auth.py

方法	路径	说明
POST	/api/auth/login	账号密码登录，返回 Token 与用户信息
GET	/api/auth/me	获取当前 Token 对应用户
POST	/api/auth/logout	撤销当前 Token
PUT	/api/auth/password	修改本人密码并撤销其他 Token

9.2 个人资料 profile.py

GET	/api/profile/me	获取本人完整资料
PUT	/api/profile/me	修改允许本人维护的资料
GET	/api/home	获取角色首页统计

9.3 基础数据 common.py

GET	/api/health	公共健康检查
GET	/api/departments	查询启用院系
GET	/api/classes	查询班级，可按院系筛选
GET	/api/semesters	查询启用学期

GET	/api/classrooms	查询教室列表
GET	/api/classrooms/<id>	查询教室详情
GET	/api/classrooms/available	按日期、节次、人数查询空闲教室
GET	/api/timetable/my	查询本人某日实际课表

9.4 课程 courses.py

GET	/api/courses	分页查询课程
GET	/api/courses/<id>	课程详情
GET	/api/courses/<id>/schedules	课程基础排课
GET	/api/enrollments/my	学生本人选课记录
POST	/api/courses/<id>/enroll	学生选课
POST	/api/courses/<id>/drop	学生退课
GET	/api/teaching/courses	教师本人授课课程
GET	/api/teaching/courses/<id>/students	教师查看课程学生
GET	/api/teaching/schedules	教师基础排课

9.5 通知 notifications.py

GET	/api/notifications	收到的通知分页列表
GET	/api/notifications/unread-count	未读数量
GET	/api/notifications/<id>	通知详情
POST	/api/notifications/<id>/read	标记已读
POST	/api/notifications	发布通知
GET	/api/notifications/published/my	本人发布记录
GET	/api/notifications/manage	教务/管理员通知管理
POST	/api/notifications/<id>/withdraw	撤回通知

9.6 教室预约 bookings.py

POST	/api/bookings	学生或教师提交预约
GET	/api/bookings/my	本人预约列表
GET	/api/bookings/<id>	预约详情
POST	/api/bookings/<id>/cancel	申请人取消
GET	/api/bookings/<id>/logs	操作日志
GET	/api/bookings/pending	待审批预约
POST	/api/bookings/<id>/approve	审批通过
POST	/api/bookings/<id>/reject	审批驳回
POST	/api/bookings/finish-expired	批量结束过期预约

9.7 请假 leaves.py

POST	/api/leaves	学生提交请假
GET	/api/leaves/my	学生本人请假
GET	/api/leaves/<id>	请假详情
POST	/api/leaves/<id>/cancel	学生取消待审批请假
GET	/api/leaves/teaching	教师查看本人课程相关请假
GET	/api/leaves/pending	待审批请假
POST	/api/leaves/<id>/approve	审批通过
POST	/api/leaves/<id>/reject	审批驳回

9.8 调课 schedule_changes.py

POST	/api/schedule-changes	教师提交调课
GET	/api/schedule-changes/my	教师本人调课记录
GET	/api/schedule-changes/<id>	调课详情
GET	/api/schedule-changes/<id>/conflicts	冲突检查结果

POST	/api/schedule-changes/<id>/cancel	教师取消待审批调课
GET	/api/schedule-changes/pending	待审批调课
POST	/api/schedule-changes/<id>/approve	审批通过
POST	/api/schedule-changes/<id>/reject	审批驳回

9.9 管理 admin.py

管理模块包括：

- 用户分页、创建、详情、修改、启停、重置密码；
- 院系创建与修改；
- 班级创建与修改；
- 教室创建与修改；
- 学期创建与修改；
- 课程创建、修改和状态调整；
- 排课创建、修改和取消；
- 审批记录查询。

所有管理接口都位于 /api/admin/...，由 ACADEMIC_STAFF 或 ADMIN 调用；部分高权限账号操作仅允许 ADMIN。

关键业务流程与规则

10.1 登录流程

账号 + 密码

- > 查询 sys_user
- > 校验密码哈希
- > 检查账号 status
- > 生成随机 Token
- > 写入 auth_token 和 expires_at
- > 返回 Token、用户 ID、姓名、角色
- > HarmonyOS SessionStore 持久化

10.2 选课流程

系统检查：

- 账号是否为正常学生；
- 课程是否启用；
- 课程是否开放；
- 当前时间是否位于选课窗口；
- 课程容量是否足够；
- 是否重复选课；
- 与已有课程是否发生学期、周次、单双周、星期和节次冲突。

退课不会删除记录，而是将状态改为 DROPPED，保留历史。

10.3 教室预约流程

学生/教师提交

- > 校验日期、节次、人数和教室状态
- > 检查课程占用
- > 检查其他预约
- > 写入 PENDING 和 SUBMIT 日志
- > 教务/管理员审批
- > 再次检查冲突
- > 更新 APPROVED/REJECTED
- > 写 approval_record 和 room_booking_log

10.4 请假流程

学生只能选择本人当前已选课程。一次请假可以关联多门课程。教师可查看涉及本人课程的请假，审批由教务或管理员执行。

状态：

PENDING -> APPROVED
-> REJECTED
-> CANCELLED

10.5 调课流程

调课类型：

- TIME：修改周次、星期和节次；
- ROOM：修改教室；
- BOTH：同时修改时间和教室。

提交和审批时都会重新检查教师冲突、教室课程冲突、教室预约冲突和基础数据有效性。课表查询不直接改写原始排课，而是计算审批通过后的实际生效安排。

10.6 通知流程

通知发布后，服务端根据接收范围生成 notification_recipient 记录。用户打开未读通知时更新 is_read 和 read_at。

教师只能撤回本人且未到截止时间的通知；教务和管理员拥有更广的撤回权限。

测试与验收

11.1 运行全接口测试

在服务端目录执行：

```
python test_all_api.py
```

测试脚本会：

1. 复制正式数据库到系统临时目录；
2. 对副本执行迁移；
3. 使用 Flask test_client() 调用接口；
4. 动态创建测试数据并覆盖成功和负向场景；
5. 检查所有 Flask 业务路由是否至少有一次成功调用；
6. 执行 SQLite 完整性检查；
7. 比较正式数据库测试前后的 SHA-256；
8. 生成或更新 API_TEST_REPORT.md；
9. 删除临时数据库。

仓库当前报告记录：

- Flask 路由：78；
- 成功路径覆盖：78/78；
- 测试用例：99；
- 通过：99；
- 失败：0；

- 正式数据库未被修改。

测试边界

该脚本属于进程内集成测试，能够验证路由、Token、角色、业务逻辑、事务和 SQLite 约束，但不覆盖真实网络、Windows 防火墙、模拟器访问、端口占用、反向代理、并发压力与长时间稳定性。因此仍需要做一次真实前后端联调。

11.2 前端冒烟测试

每次合并重要代码后至少验证：

1. 健康检查；
2. 四种角色登录；
3. 关闭应用后重新打开，Token 能恢复；
4. Token 失效后自动回登录状态；
5. 学生课程查询、选课和退课；
6. 学生教室申请和请假；
7. 教师课程名单、教室申请和调课；
8. 教务审批教室、请假和调课；
9. 教师/教务发布通知；
10. 管理员创建用户和基础数据；
11. 课表能显示审批通过的调课结果。

11.3 DevEco 构建验收

- 执行 Build HAP(s)；
- 确认没有 ArkTSCheck 错误；
- 确认导入路径大小写完全一致；
- 确认对象字面量均对应显式接口或类；
- 确认 StatusPill 使用参数名 pillColor；
- 确认根工程而非 1/ 工程被运行。

常见问题与排查

12.1 无法连接 Flask

按顺序检查：

1. Flask 终端是否仍在运行；
2. 浏览器能否打开 `http://127.0.0.1:8080/api/health`；
3. `ApiConfig.ets` 是否使用电脑当前 IPv4；
4. 是否写了 `/api`；
5. Windows 防火墙是否允许 Python 或 8080；
6. 模拟器和电脑网络是否正常；
7. 电脑 IP 是否因切换 Wi-Fi 而变化；
8. `module.json5` 是否存在 INTERNET 权限。

12.2 登录提示网络失败

若健康检查可用但登录失败：

- 检查账号是否存在；
- 检查测试密码是否为 `Test123456`；
- 检查账号是否被停用；
- 检查数据库是否为正确文件；
- 查看 Flask 终端日志和 HTTP 状态码。

12.3 提示 SessionStore 尚未初始化

原因通常是页面在 `Api.init(this.context)` 完成前调用了 API。确认 `EntryAbility.ets` 先初始化，再加载 `pages/Index`。

12.4 修改代码但应用没变化

重点检查是否误改了 `1/entry/`，而运行的是根目录 `entry/`。也可以查看 DevEco 工程根目录与 Build 模块路径。

12.5 端口 8080 被占用

PowerShell 查询：

```
netstat -ano | findstr :8080
```

可以停止占用进程，或统一修改 Flask 端口和 `ApiConfig.ets`。两端必须一致。

12.6 数据库被锁定

SQLite 已设置 15 秒等待，但大量并发写入仍可能锁定。检查：

- 是否有数据库管理工具长期占用写事务；
- 写操作是否正确 `commit` 或 `rollback`；
- 是否同时启动多个开发服务写同一个数据库；
- 是否在网络共享目录中直接运行 SQLite。

12.7 JSON 字段为空或类型不对

前端 Model 字段必须和 Flask 返回的 `camelCase` 完全一致。后端 `api_utils.py` 会把数据库的 `snake_case` 转换为 `camelCase`，例如：

```
real_name      -> realName  
classroom_id   -> classroomId  
created_at     -> createdAt
```

当前初版的已知限制与改进建议

本节用于避免使用者把当前初版误认为已完成的生产系统。

13.1 仓库存在两套 HarmonyOS 工程

根目录和 1/ 都包含工程配置。建议后续确认保留哪一套，将另一套移动到独立分支或归档目录，避免团队成员误改。

13.2 ApiConfig 写死局域网 IP

当前 ApiConfig.ets 中的 IP 只适用于某台开发电脑。团队成员拉取代码后必须修改。后续可使用：

- debug/release 多环境配置；
- 本地不提交的配置文件；
- 固定测试服务器域名。

13.3 ApiClient 的请求头类型

当前 ApiClient 使用 Record<string, string> 构造请求头。在部分 ArkTS 严格检查环境中可能产生动态对象类型警告。若 DevEco 报错，应改为显式声明的请求头接口。

13.4 ApiError 未保留冲突详情

Flask 在教室、排课和调课冲突时可能在错误响应 data 中返回具体冲突对象，但当前 ApiError 只保存 code、message 和 httpStatus。前端目前只能显示文字，不能展开冲突明细。

13.5 教师请假查看页面尚未接入

后端和 LeaveApi 已提供教师查看本人课程相关请假的接口，但当前 WorkflowPage.ets 的教师标签主要展示教室申请和调课申请，没有独立呈现学生请假情况。后续应补充教师请假列表。

13.6 管理页面仍偏演示型

当前 AdminPage.ets 已能创建与启停多类数据，但表单仍使用直接填写 ID 的方式，且部分更新能力未形成完整交互。后续应改为选择器、详情编辑页和二次确认弹窗。

13.7 初始化失败没有备用页面

EntryAbility.ets 中若 Api.init() 失败，目前只记录日志，不继续加载页面。后续建议增加初始化失败页或重试机制。

13.8 后端开发模式不适合生产

当前 Flask 启动使用 debug 模式和 HTTP，Token 也通过明文 HTTP 发送。正式环境必须改用 HTTPS、生产 WSGI 和安全配置。

团队开发建议

14.1 推荐分工边界

角色	负责内容
前端页面开发	只调用 Api，维护页面状态、组件和交互，不直接访问数据库
API 封装维护	维护 api/core、api/model、api/module，确保与 Flask 契约一致
后端开发	维护 Blueprint、权限、事务、冲突校验和数据库结构
测试	维护 test_all_api.py、前端冒烟测试和测试报告
数据与文档	管理测试数据、数据库备份、接口文档和演示账号

14.2 修改接口的标准流程

1. 先修改 Flask 路由和业务逻辑；
2. 更新或新增后端自动化测试；
3. 运行 `python test_all_api.py`；
4. 更新 ArkTS Model；
5. 更新对应业务 Api 模块；
6. 在页面中调用；
7. 执行 DevEco 全量构建；
8. 使用真实模拟器联调；
9. 更新项目说明和接口说明。

14.3 提交前检查清单

- ✓没有提交真实密码、私钥和生产 Token；
- ✓没有误提交个人绝对路径；
- ✓没有修改错误的 1/ 目录；
- ✓Flask 测试全部通过；
- ✓DevEco Build HAP(s) 通过；
- ✓ApiConfig.ets 的地址已和团队约定一致；
- ✓数据库结构变化已同步 migrate.py；
- ✓新接口具有角色校验；
- ✓多表写入使用事务；
- ✓前端页面只通过统一 Api 调用。

常用命令速查

A.1 后端

```
# 创建虚拟环境
python -m venv .venv

# PowerShell 激活
.\.venv\Scripts\Activate.ps1

# 安装依赖
python -m pip install -r requirements.txt

# 迁移数据库
python migrate.py

# 创建管理员
python create_admin.py --account admin --password Admin123456 --name
↪ 系统管理员

# 生成测试数据（危险：清空现有数据）
python seed_data.py --users 2000 --password Test123456 --replace

# 启动服务
python app.py

# 全接口测试
python test_all_api.py
```

A.2 Windows 网络

```
# 查询电脑 IPv4
ipconfig
```

```
# 查询 8080 端口占用
netstat -ano | findstr :8080

# 健康检查
Invoke-RestMethod http://127.0.0.1:8080/api/health
```

角色功能验收表

功能	学生	教师	教务	管理员	备注
登录与会话恢复	✓	✓	✓	✓	所有账号预建
查看首页统计	✓	✓	✓	✓	统计内容按角色变化
查看个人课表	✓	✓	✓	✓	管理角色可查看全量 计算结果接口
课程查询	✓	✓	✓	✓	页面展示按角色变化
选课/退课	✓	✗	✗	✗	仅学生
查看课程学生名单	✗	✓	✗	✗	教师仅本人课程
教室申请	✓	✓	✗	✗	教务和管理员负责审 批
请假申请	✓	✗	✗	✗	仅学生
查看相关学生请假	✗	✓	✓	✓	教师 UI 尚待补充
调课申请	✗	✓	✗	✗	教师仅本人排课
审批	✗	✗	✓	✓	教室、请假、调课
发布通知	✗	✓	✓	✓	不提供学生发布
人员与基础数据管理	✗	✗	✓	✓	高权限操作受角色限 制
修改个人资料和密码	✓	✓	✓	✓	权威身份字段不可由 普通用户修改

文档维护说明

当仓库结构、启动方式、测试账号、数据库迁移、SDK 版本或接口发生变化时，应更新本说明。建议在文档封面同步记录分支和提交 SHA，避免说明与代码版本不一致。

最终启动顺序

1. 打开仓库根目录；
2. 进入 `teaching_flask_server/`；
3. 激活虚拟环境并安装依赖；
4. 确认 `data/teaching.db` 存在；
5. 执行 `python migrate.py`；
6. 执行 `python app.py`；
7. 浏览器检查 `/api/health`；
8. 修改 `ApiConfig.ets` 为电脑局域网 IP；
9. 在 DevEco Studio 运行根目录 `entry` 模块；
10. 使用测试账号登录并完成角色流程验收。